

ROS2

DAY 5 – Manual 2

ROS2 Actions – Implementation

Complete Beginner-Friendly

Objective

By the end of this manual, you will be able to:

1. Write a ROS2 **Action Server** node.
2. Write a ROS2 **Action Client** node.
3. Understand **each line of the code**.
4. Run and test a ROS2 action with **goal, feedback, and result**.

Step 0 – Pre-Requisites

- Manual 1 completed (package created, `.action` file ready, workspace built)
- ROS2 sourced:

```
source ~/ros2_ws/install/setup.bash
```

Step 1 – Create Action Server Node

1. Navigate to the package folder:

```
cd ~/ros2_ws/src/day5_actions/day5_actions
```

2. Create `move_server.py`:

```
nano move_server.py
```

3. Paste the code:

```
import rclpy
from rclpy.node import Node
from rclpy.action import ActionServer
from day5_actions.action import MoveRobot
from rclpy.executors import MultiThreadedExecutor
import time
```

Explanation:

- Imports ROS2 Python libraries, action interface, and time module
-

4. Define the server node:

```
class MoveServer(Node):
    def __init__(self):
        super().__init__('move_server')
        self._action_server = ActionServer(
            self,
            MoveRobot,
            'move_robot',
            self.execute_callback
        )
        self.get_logger().info("MoveRobot Action Server Ready")
```

Explanation:

- Creates node `move_server`
 - Registers `MoveRobot` action on topic `move_robot`
 - `execute_callback` handles goal execution
-

5. Define the callback function:

```
def execute_callback(self, goal_handle):
    distance = goal_handle.request.distance
    self.get_logger().info(f"Received goal: Move {distance} meters")

    feedback_msg = MoveRobot.Feedback()
    current_distance = 0.0

    while current_distance < distance:
        current_distance += 1.0
        feedback_msg.current_distance = current_distance
        goal_handle.publish_feedback(feedback_msg)
        self.get_logger().info(f"Moving...
{current_distance}/{distance}")
        time.sleep(1)

    goal_handle.succeed()

    result = MoveRobot.Result()
    result.result = f"Reached {distance} meters"
    return result
```

Explanation:

- Reads goal distance
 - Publishes **feedback** every second
 - Marks goal as succeeded and sends **result**
-

6. Add main function:

```
def main():
    rclpy.init()
    node = MoveServer()
    rclpy.spin(node)
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

Explanation:

- Initializes ROS2
 - Spins server until terminated
-

Step 2 – Create Action Client Node

1. Create move_client.py:

```
nano move_client.py
```

2. Paste the code:

```
import rclpy
from rclpy.node import Node
from rclpy.action import ActionClient
from day5_actions.action import MoveRobot
```

3. Define the client node:

```
class MoveClient(Node):
    def __init__(self):
        super().__init__('move_client')
        self._action_client = ActionClient(self, MoveRobot, 'move_robot')

    def send_goal(self, distance):
        self._action_client.wait_for_server()
```

```

        goal_msg = MoveRobot.Goal()
        goal_msg.distance = distance
        self._send_goal_future = self._action_client.send_goal_async(
            goal_msg,
            feedback_callback=self.feedback_callback
        )
        self._send_goal_future.add_done_callback(self.goal_response_callback)

    def feedback_callback(self, feedback_msg):
        self.get_logger().info(f"Feedback: {feedback_msg.feedback.current_distance}")

    def goal_response_callback(self, future):
        goal_handle = future.result()
        if not goal_handle.accepted:
            self.get_logger().info("Goal rejected")
            return
        self._get_result_future = goal_handle.get_result_async()
        self._get_result_future.add_done_callback(self.get_result_callback)

    def get_result_callback(self, future):
        result = future.result().result
        self.get_logger().info(f"Result: {result}")

```

Explanation:

- Connects to server using ActionClient
 - Sends goal distance
 - Receives feedback in real-time
 - Prints final result
-

4. Add main function:

```

def main():
    rclpy.init()
    node = MoveClient()
    node.send_goal(5.0) # Move 5 meters
    rclpy.spin(node)
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

Explanation:

- Sends a goal of 5 meters to server
 - Spins client to receive feedback and result
-

Step 3 – Run and Test the Action

Step 3.1 – Start Server

Open terminal:

```
ros2 run day5_actions move_server
```

You will see:

```
[INFO] [move_server]: MoveRobot Action Server Ready
```

Step 3.2 – Start Client

Open another terminal:

```
ros2 run day5_actions move_client
```

Client terminal shows feedback every second:

```
[INFO] [move_client]: Feedback: 1.0  
[INFO] [move_client]: Feedback: 2.0  
...  
[INFO] [move_client]: Result: Reached 5.0 meters
```

Server terminal shows:

```
[INFO] [move_server]: Received goal: Move 5 meters  
[INFO] [move_server]: Moving... 1/5  
...  
[INFO] [move_server]: Moving... 5/5
```

Step 4 – Concepts Recap

- **Actions** allow asynchronous long-running tasks with feedback
 - **Server** executes goal, provides feedback, and sends result
 - **Client** sends goal, receives feedback, and gets final result
 - `.action` file defines **goal, feedback, and result structure**
 - Beginner-friendly step-by-step implementation
-